Data Cleaning & Preprocessing Explanation

This code prepares the raw salary dataset for accurate regression analysis. Raw data often contains errors, inconsistencies, missing values, and outliers. Cleaning ensures reliable and unbiased results.

Load Data

df = pd.read_csv("salary_data.csv")

- The dataset is loaded into a dataframe.
- df.shape shows number of rows and columns.
- This gives an overview of dataset size.

Purpose:
To begin preprocessing using structured data.

Clean Column Names

df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

This step:

- Removes extra spaces
- Converts column names to lowercase
- Replaces spaces with underscores

Example:

- "Education Level" → "education_level"

Purpose:
Standardized column names prevent coding errors and improve readability.

Check Data Types

df.info()

This shows:

- Data types (int, float, object)
- Missing values
- Number of non-null entries

Purpose:
To verify whether variables are correctly formatted (e.g., salary should be numeric).

Clean Salary Column

df['salary'] = df['salary'].replace('[₹,]', '', regex=True)

df['salary'] = pd.to_numeric(df['salary'], errors='coerce')

What this does:

- Removes ₹ symbol and commas
- Converts salary to numeric format
- Invalid values become NaN

Then:

df['salary'] = df['salary'].fillna(df['salary'].median())

Missing salary values are replaced with the **median**.

Why median?

- Median is robust to extreme values.
- Prevents distortion caused by outliers.

Remove Other Missing Values

df.dropna(inplace=True)

Removes rows with missing values in other columns.

Purpose:
Ensures complete data for regression analysis.

Remove Outliers (IQR Method)

You used the **Interquartile Range (IQR)** method:

Formula:
IQR = Q3 - Q1

Lower bound:
Q1 − 1.5 × IQR

Upper bound:
Q3 + 1.5 × IQR

This removes extreme salary values that are unusually high or low.

Also:

df = df[df['salary'] >= 10000]

Removes unrealistic low salaries.

Purpose:
Outliers can distort regression coefficients and reduce model accuracy.

Standardize Gender

```
df['gender'] = df['gender'].str.strip().str.lower()
```

This:

- Removes spaces
- Converts to lowercase

Then:

```
df['gender'].replace({'m': 'male', 'f': 'female'})
```

Standardizes gender categories.

You also removed "other":

```
df = df[df['gender'].isin(['male', 'female'])]
```

Purpose:
Ensures consistent categories for dummy encoding.

Standardize Education Level

Education entries may vary like:

- "Bachelor's"
- "bachelor degree"
- "Bachelor's Degree"

You standardized all into:

- bachelors
- masters
- phd
- high_school

Purpose:
Prevents duplicate categories during dummy encoding.

Remove Duplicates

```
df.drop_duplicates(inplace=True)
```

Removes repeated rows.

Purpose:
Avoids biased results from duplicate entries.

Final Data Check

df.info()

df.describe()

- Confirms clean dataset
- Shows summary statistics
- Ensures correct data types

Purpose:
Final verification before model building.

Save Cleaned Dataset

df.to_csv("cleaned_salary_dataset_v1.csv")

Saves the cleaned dataset for:

- Dummy variable creation
- Regression modeling
- Future reproducibility

Why Data Cleaning Is Important in Your Project

Without cleaning:

- Salary symbols would cause errors
- Missing values would distort results
- Outliers would bias regression
- Inconsistent categories would create wrong dummy variables

Proper preprocessing ensures that the gender pay gap analysis is accurate and statistically valid.

“We cleaned the dataset by removing missing values, correcting salary format, eliminating outliers using IQR method, standardizing gender and education categories, and removing duplicates. This ensured reliable regression results and prevented biased estimation of the gender pay gap.”